DID & Stakeholder Specification

Section 1: The Four Roles

Role	Smart Contract Name	Human-Readable Label	Examples
Holder	Holder	Attendee	Student, professional, event participant
Issuer	Issuer	Credential Issuer	Professor, event organizer, webinar host, guest lecturer
Verifier	Verifier	Verifier	Recruiter, institution, employer, admin
Admin	Admin	System Admin	Contract deployer; set at deployment, cannot be changed

Key rules everyone must respect:

- No one can self-assign the Issuer role; Admin must assign it manually
- Admin is the wallet that deploys the contract; hardcoded as owner
- Verifying a credential requires no role, it is a public read operation
- A wallet that has not called registerDID has role = None and cannot perform any action except register

Section 2: What Each Role Can Do

Holder (Attendee)

- Connect MetaMask and call registerDID
- View their own credentials by wallet address
- Share a credential ID with a verifier to prove attendance
- Cannot create sessions, issue credentials, or assign roles

Issuer (Credential Issuer)

- Must be assigned Issuer role by Admin before doing anything
- Create a session on-chain (title, description, date, event type)
- Select a holder wallet address and issue a credential to them

- Revoke only credentials they originally issued
- Cannot revoke another issuer's credentials
- Event type covers: university class, webinar, guest lecture, workshop, conference, or any attendance-based event

Verifier

- Enter a credential ID into the frontend
- Fetch on-chain hash and revocation status
- Fetch full credential JSON from IPFS and hash it locally
- Compare local hash to on-chain hash to confirm authenticity
- No write access to anything on-chain, it’s purely read operations

Admin

- Deploys both contracts (DIDRegistry and CredentialRegistry)
- Assigns Issuer role to specific wallet addresses
- Can deactivate a DID if required
- Only caller allowed for assignRole and deactivateDID

Section 3: Role-Permission Matrix

Function	Holder	Issuer	Verifier	Admin
registerDID	✅	✅	✅	✅
assignRole	❌	❌	❌	✅ only
deactivateDID	❌	❌	❌	✅ only
createSession	❌	✅	❌	❌
issueCredential	❌	✅	❌	❌
revokeCredential	❌	✅ own only	❌	❌
verifyCredential	✅	✅	✅	✅
viewMyCredentials	✅ own only	❌	❌	❌
getCredentialStatus	✅	✅	✅	✅

Section 4: Identity Lifecycle Flows

Flow 1- DID Registration

1. Attendee opens React app
2. Clicks "Connect Wallet" → MetaMask popup appears
3. Attendee approves → app captures wallet address
4. Attendee clicks "Register DID"
5. App calls DIDRegistry.registerDID()
6. Contract checks: this address is not already registered → revert if duplicate
7. Contract generates DID string: did:ethr:0x<walletAddress>
8. Stores DID struct on-chain, auto-assigns Holder role
9. Emits DIDRegistered event- no PII, only address and DID string
10. Frontend displays: "Your DID is registered: did:ethr:0x..."

Flow 2- Session Creation and Credential Issuance

1. Credential Issuer connects MetaMask, contract checks Issuer role on load
2. Clicks "Create Session" → fills title, description, date, and event type
3. App calls CredentialRegistry.createSession(title, description, date, eventType)
4. Contract stores session on-chain, emits SessionCreated
5. Issuer selects a holder wallet address from input field
6. App builds credential JSON off-chain using Mohan's schema
7. App pins JSON to IPFS → receives back IPFS URI
8. App hashes full JSON with keccak256 → produces credential hash
9. App calls issueCredential(holderAddress, sessionId, credentialHash, ipfsURI)
10. Contract stores hash + Active status + IPFS URI on-chain
11. Emits CredentialIssued- credential ID, holder address, issuer address, session ID; no PII
12. Frontend displays: "Credential issued successfully"

Flow 3- Credential Verification

1. Verifier connects MetaMask, no role check needed
2. Enters credential ID into verification input field
3. App calls contract → fetches on-chain: credential hash, status, IPFS URI
4. If status is Revoked → immediately display "Invalid: Credential has been revoked ❌" and stop
5. App fetches full credential JSON from IPFS using the URI
6. App hashes the fetched JSON locally with keccak256
7. App compares local hash to on-chain hash
8. If hashes match → display "Valid ✅; Credential is authentic"
9. If hashes do not match → display "Invalid: Data has been tampered ❌"

Flow 4- Credential Revocation

1. Credential Issuer connects MetaMask
2. Navigates to their issued credentials list
3. Selects a credential and clicks "Revoke"
4. Contract checks: msg.sender == original issuer stored in credential struct → revert if not
5. Contract changes credential status: Active → Revoked
6. Emits CredentialRevoked- credential ID, issuer address, timestamp
7. Frontend displays: "Credential successfully revoked"

Section 5: Events Reference

All events must contain zero PII, no names, emails, or personal data. Only addresses, IDs, hashes, and timestamps.

Event	Emitted When	Parameters
DIDRegistered	New DID registered	address, didString, timestamp
RoleAssigned	Admin assigns a role	address, role, timestamp
DIDDeactivated	Admin deactivates a DID	address, timestamp

SessionCreated	Issuer creates a session	sessionId, issuerAddress, eventType, timestamp
CredentialIssued	Credential issued to attendee	credentialId, holderAddress, issuerAddress, sessionId
CredentialRevoked	Issuer revokes a credential	credentialId, issuerAddress, timestamp
CredentialVerified	Verification attempted	credentialId, verifierAddress, result, timestamp

Section 6: Note on System Scope

This system is context-agnostic. The same smart contracts work across:

- University classes and lectures
- Professional webinars
- Guest lectures and seminars
- Workshops and conferences
- Any event requiring attendance certification

The eventType field in createSession captures the context without requiring any contract changes. This is a deliberate design choice that makes the system reusable across institutions and industries.

Section 7: What Each Person Must Do with This Document

→ Kinjal

Use Section 3 to write every modifier: onlyAdmin, onlyIssuer, onlyRegistered. Use Section 5 to define all events in both contracts. Use Section 2 to decide which functions go in which contract. Note the new eventType parameter in createSession.

→ Siva

Use Section 3 for every require statement. Use Flows 1–4 in Section 4 for the exact logic order inside each function. The revocation check in Flow 4 Step 4 is your most critical require check.

→ Mohan

Use Flow 2 Steps 6–8 to understand exactly where your credential JSON fits in the issuance process. Add eventType as a field in the credential JSON schema since sessions now carry event type. Your field names are the input to keccak256 hashing, finalize them and share the schema back with the team before Siva writes the hashing logic.

→ Akshith

Use Section 4 flows as your page-by-page frontend blueprint. Flow 3 is your most critical implementation. Use Section 2 to decide what UI elements to show or hide based on the connected wallet's role, Holders should never see the "Create Session" button. The session creation form now has an Event Type dropdown field.